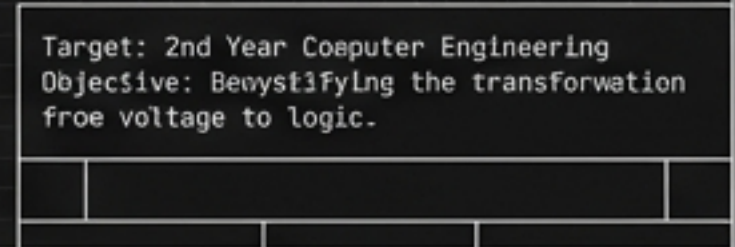
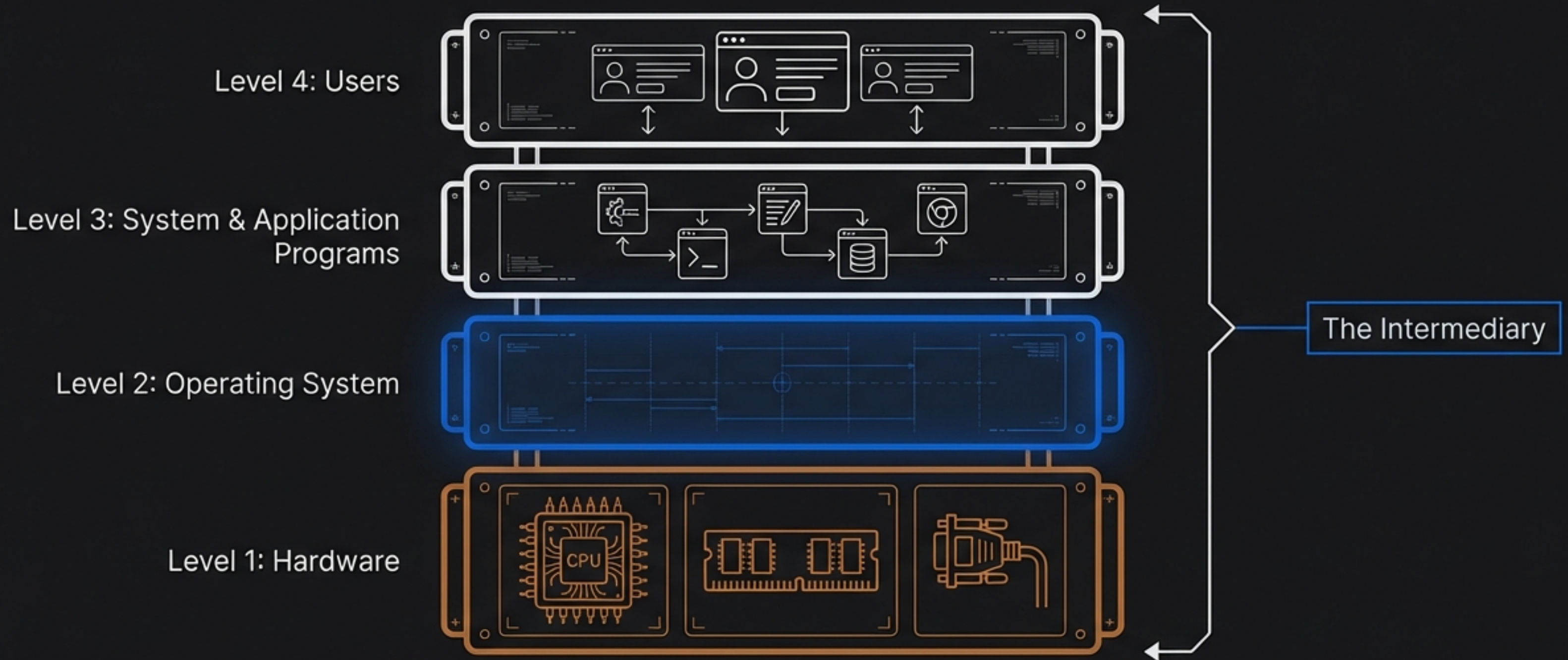


Operating Systems Architecture & Mechanics



The Hierarchy of Abstraction

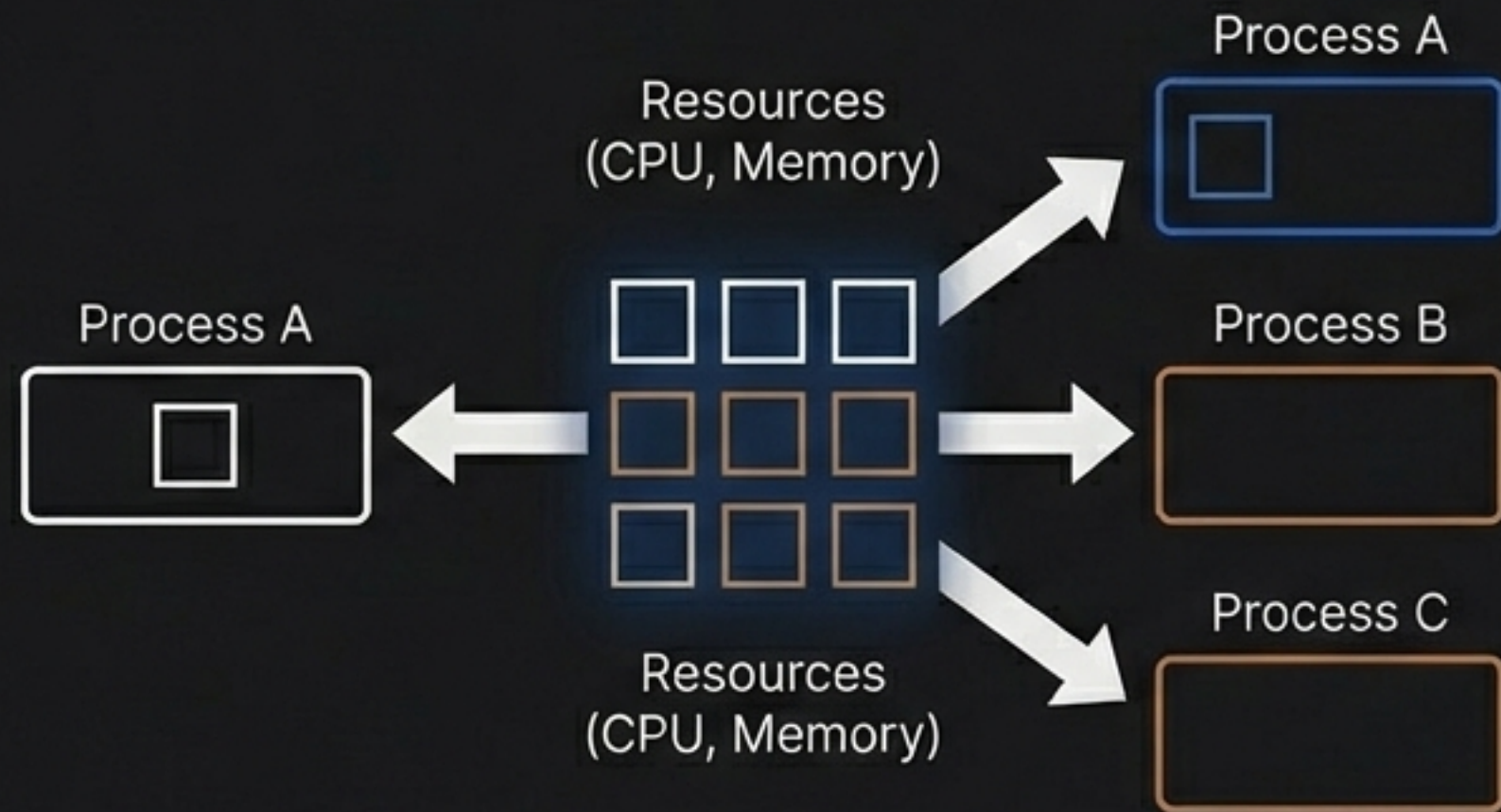


The OS acts as the Intermediary between the user and the bare metal.

Goal: Ease of use balanced with efficient hardware operation.

The Dual Mandate: Allocator and Controller

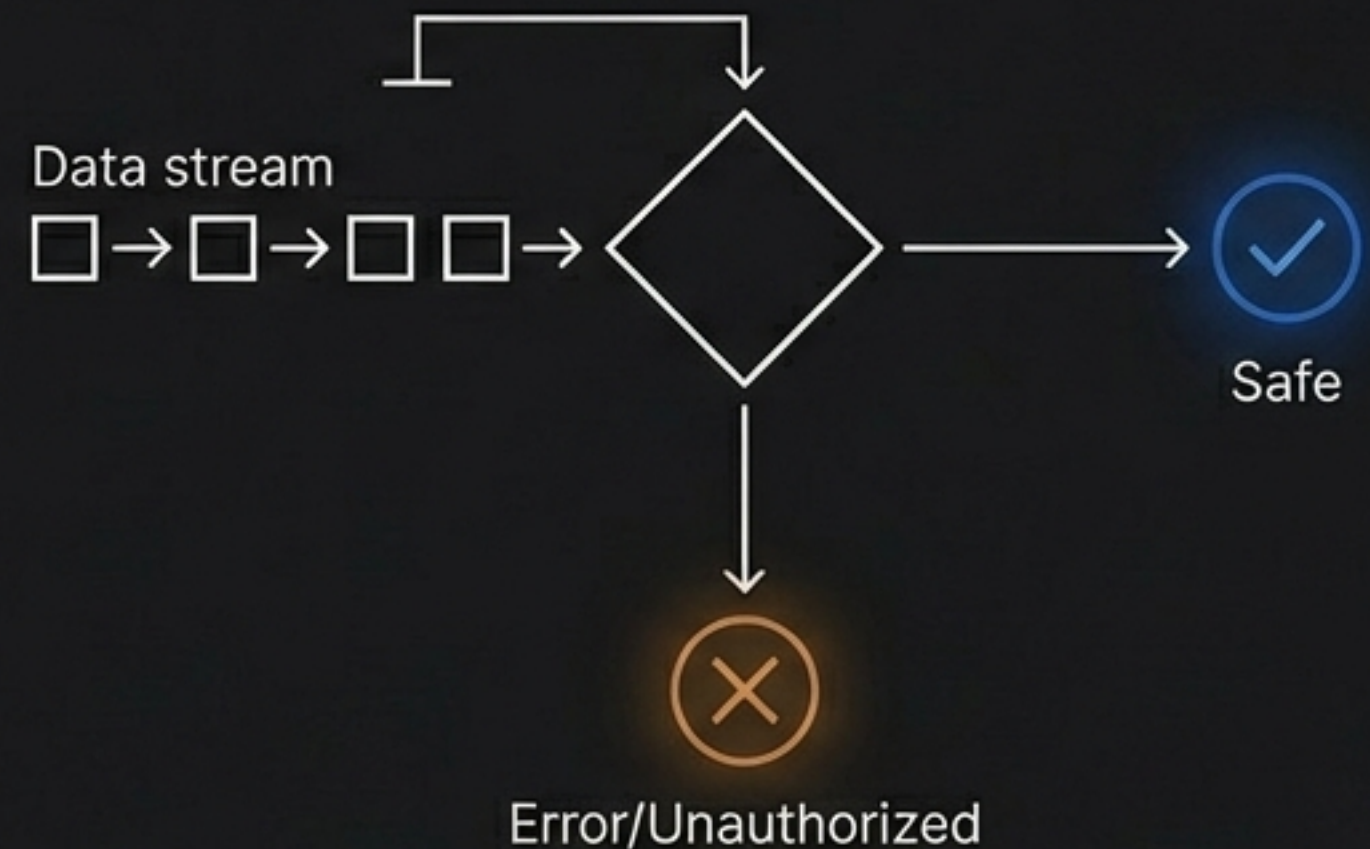
The Allocator



Resource Allocator

Manages conflicting requests for limited hardware resources (fair and efficient).

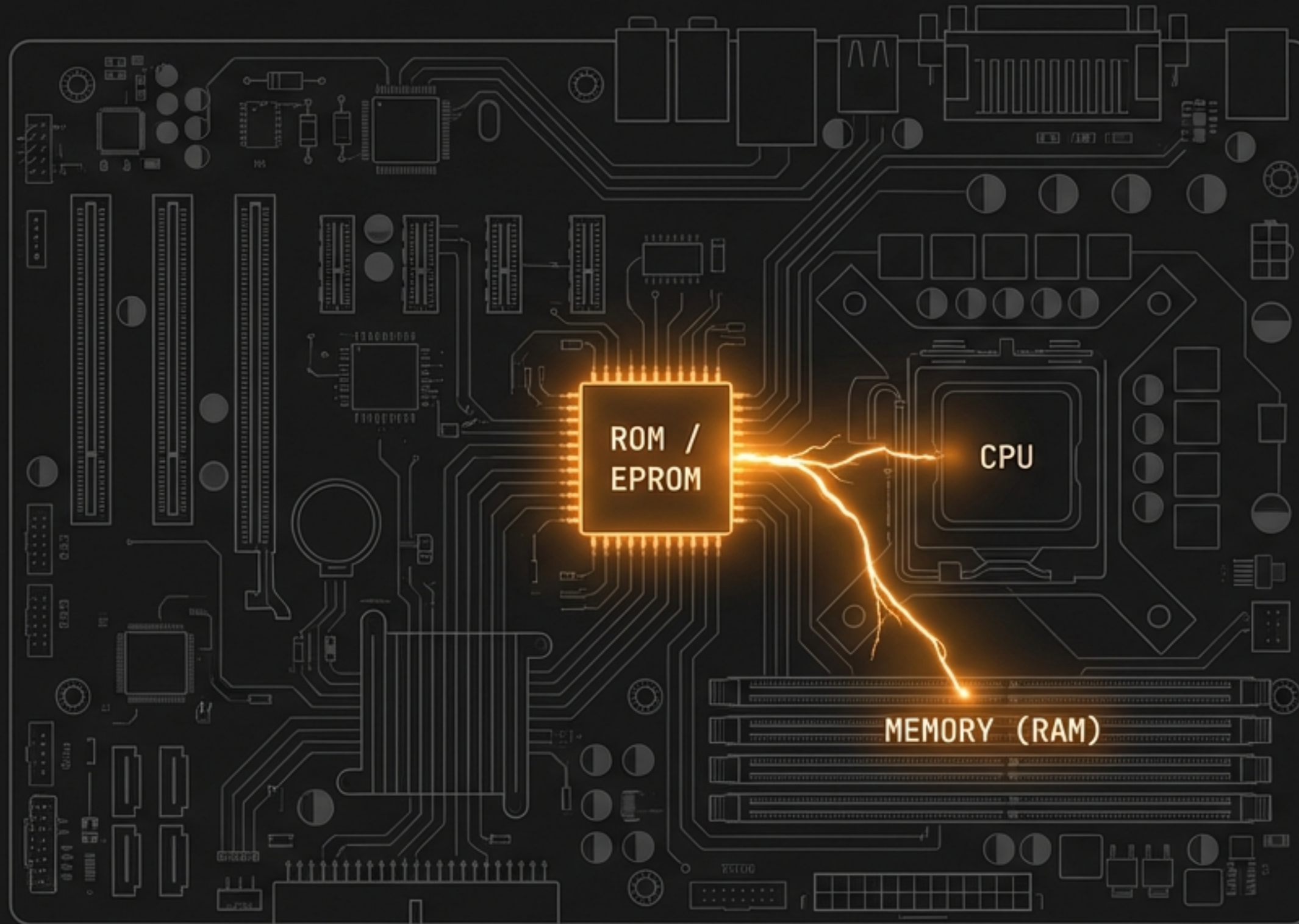
The Controller



Control Program

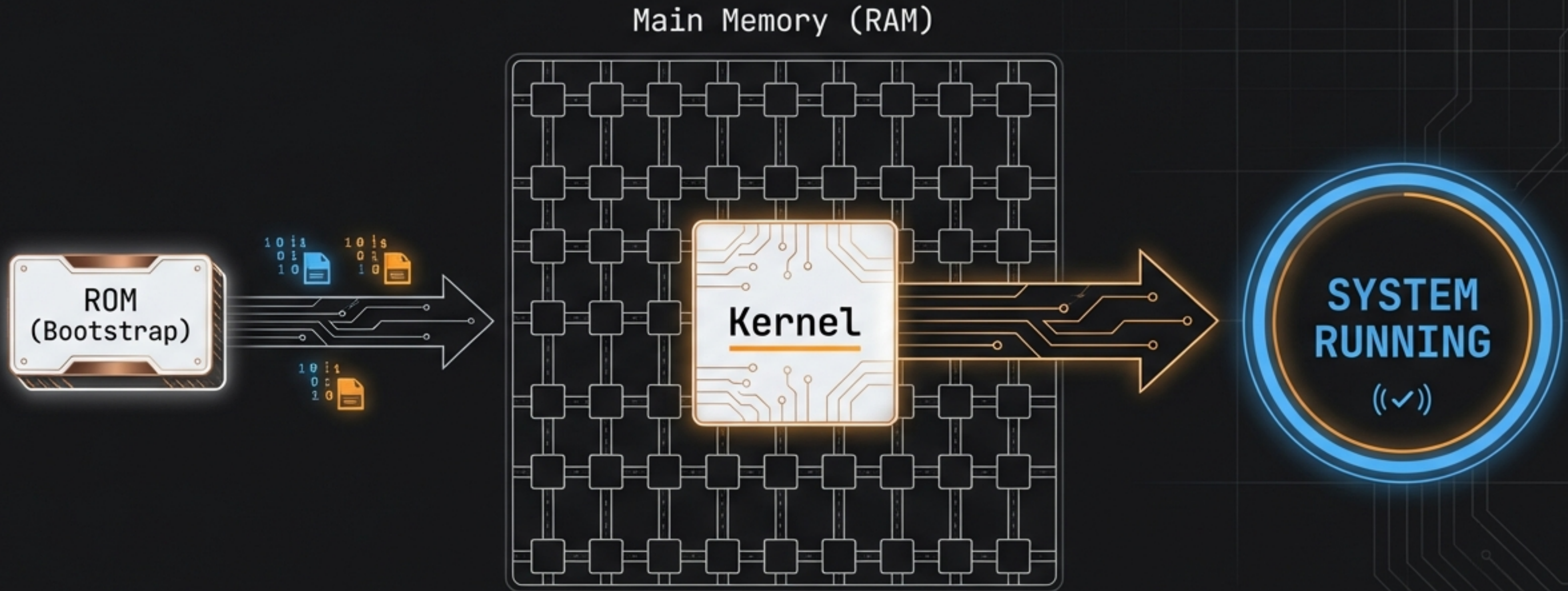
Supervises execution to prevent errors and improper use of the computer.

Ignition: The Bootstrap Program



- The Problem: Volatile memory (RAM) is empty at power-up.
- The Solution: Firmware.
- Location: ROM or EPROM.
- Task: Initialize system components (CPU registers, device controllers, memory).

The Handoff: ROM to RAM



1. Power On.
2. Bootstrap Initializes Hardware.

3. Load OS Kernel into Main Memory.
4. Transfer execution control to the Kernel.

The system is now awake and waiting.

The Reflex: Interrupt-Driven Architecture

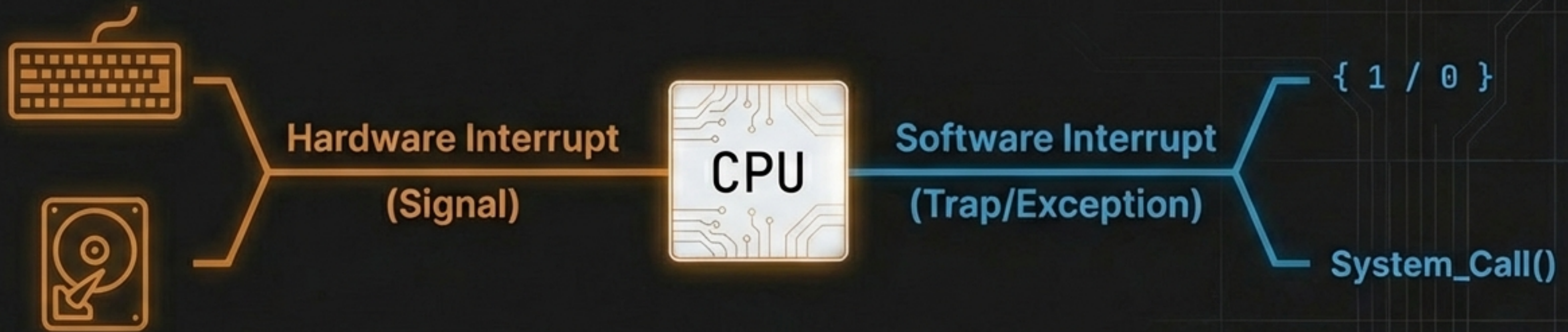
The OS does not poll;
it reacts.

CPU stops execution to
handle event via ISR.



Interrupt Driven:
Event triggers transfer
of control.

Triggers: Hardware Signals vs. Software Traps



Hardware Interrupts:

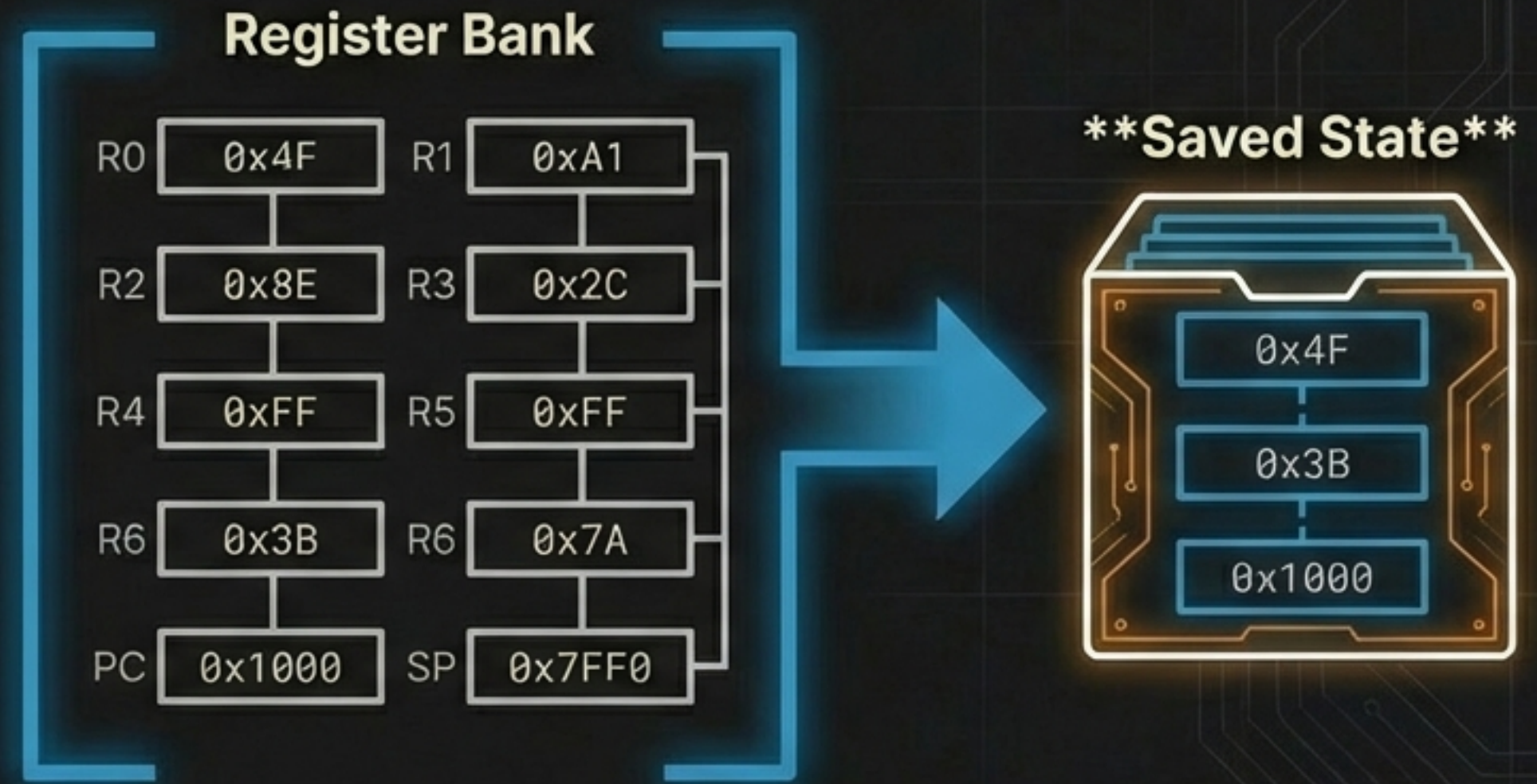
Signals sent by devices via system bus.

Software Interrupts (Traps):

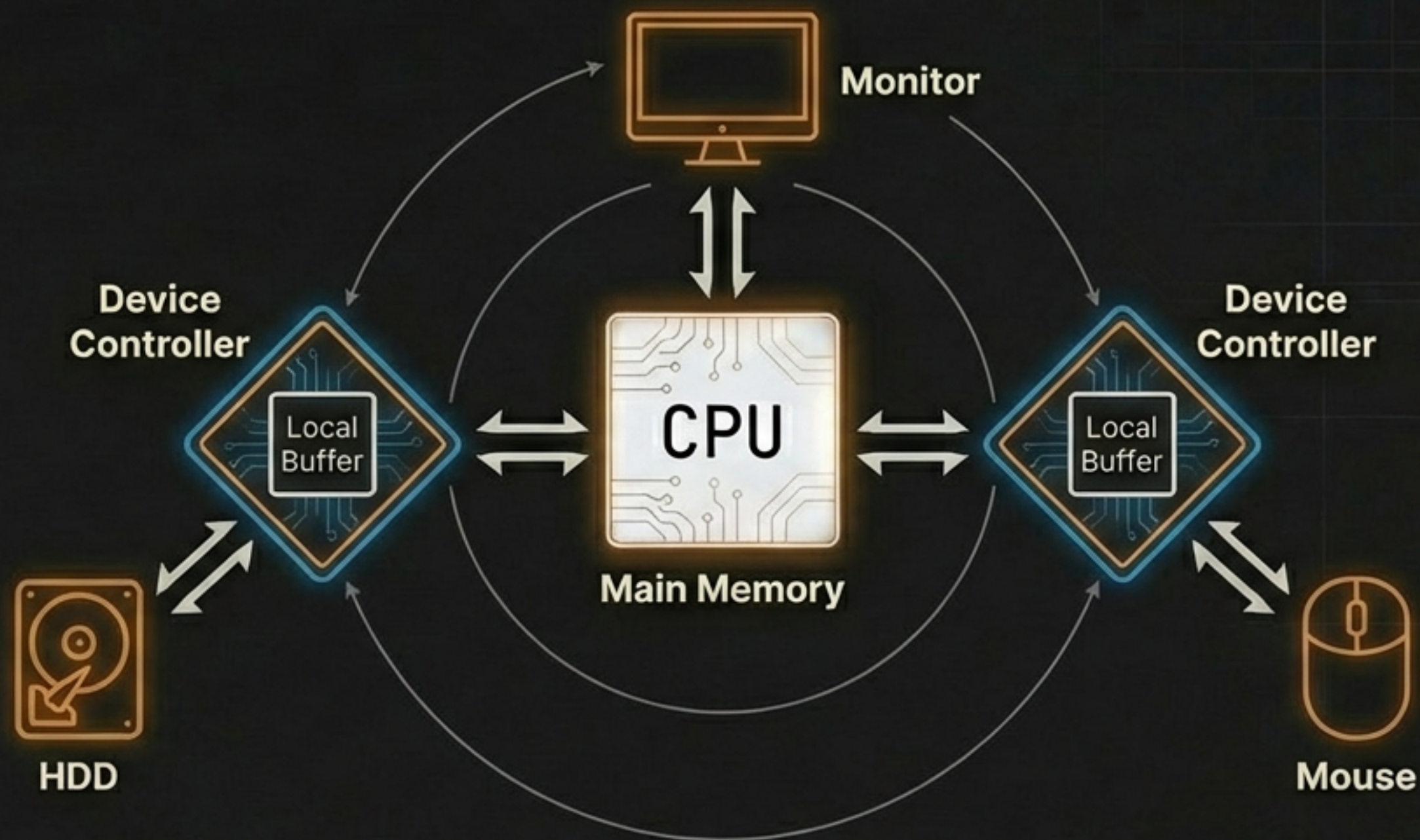
- Errors (e.g., division by zero)
- User requests for OS services (System Calls)

Preserving State: The Context Switch

- **Challenge:** Handling an interrupt must not kill the current process.
- └ **Mechanism:** The CPU saves its Current State before jumping to the ISR.
- └ **Saved Data:** Registers + Program Counter.
- └ **Result:** Seamless return to the user program.



I/O Topology: The Conversation

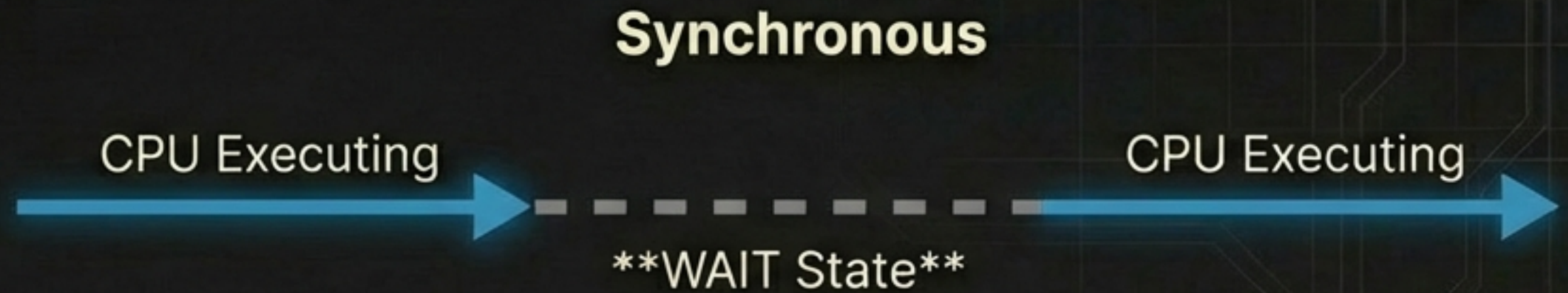


- 1. Devices have specific Device Controllers.
- 2. Each controller has a Local Buffer.
- 3. CPU moves data: Main Memory $\leftrightarrow$ Local Buffer.
- 4. Controller moves data: Local Buffer $\leftrightarrow$ Device.

Concurrency: Synchronous vs. Asynchronous I/O

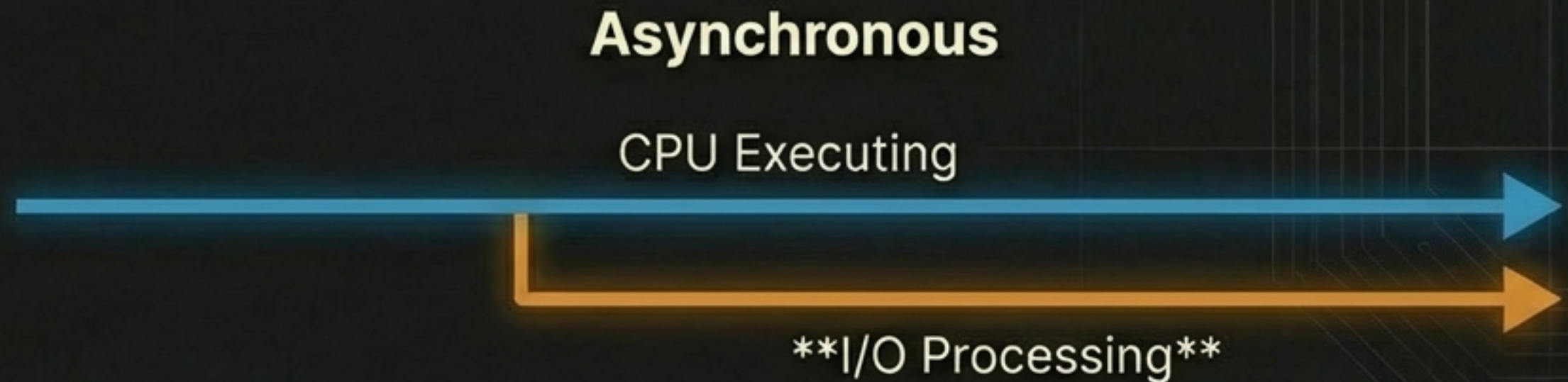
Synchronous:

CPU Idle = Inefficient.
Control returns only
after I/O completes.



Asynchronous:

Concurrent Processing
= Efficient.
Control returns
immediately.



Tracking Chaos: The Device Status Table

The OS needs a map of all activity.

Device Status Table tracks:

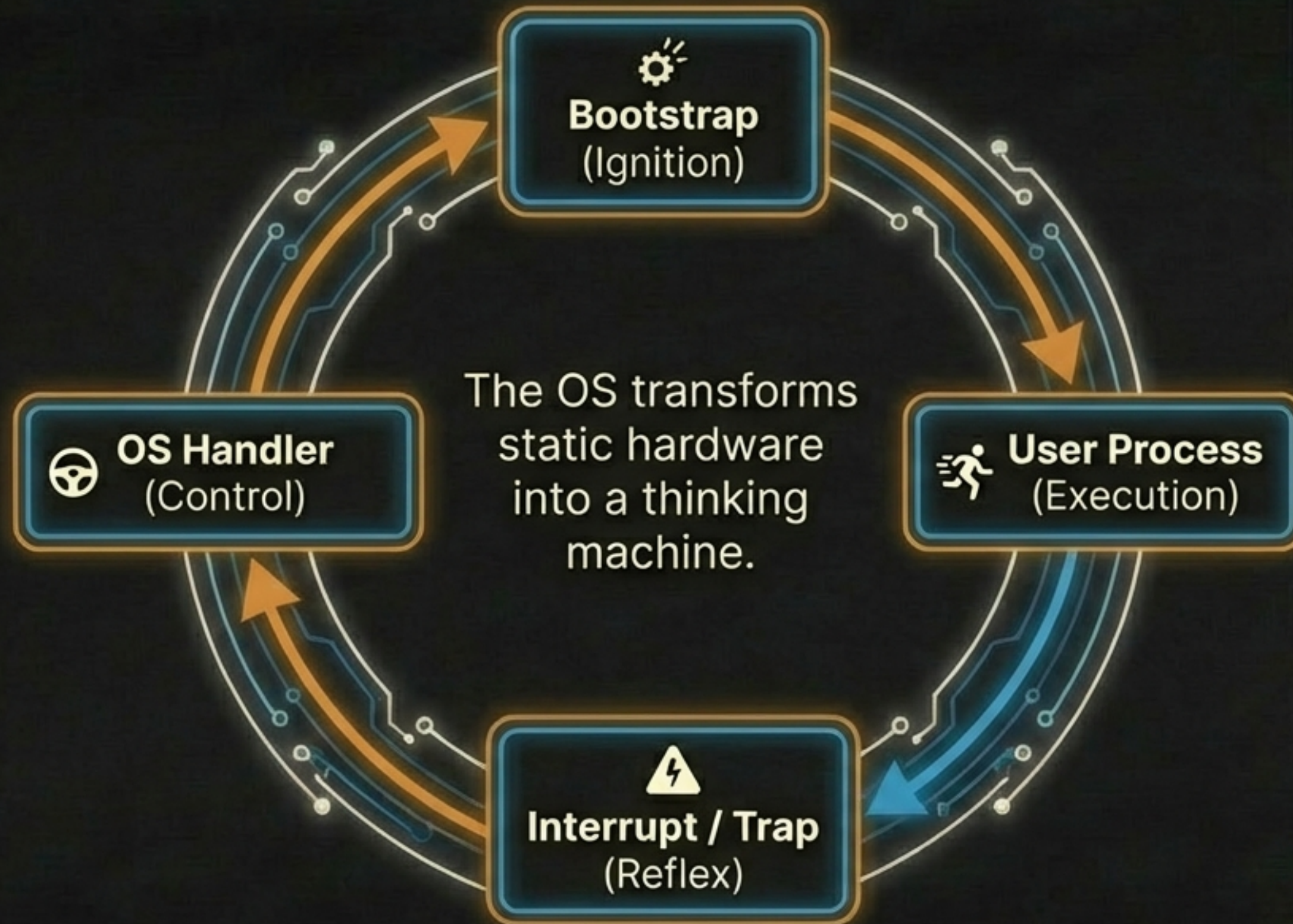
- Device Type
- Address
- State (Busy, Idle, Error)

Purpose: Ensures no two processes accidentally write to the same device.

The diagram shows a table with four columns: Device ID, Type, Address, and Status. It contains three rows of data. The table is highlighted with a blue border and an orange glow. Arrows point from the left towards the table, indicating the OS's access to the data.

Device ID	Type	Address	Status
001	Keyboard	0x3F	IDLE
002	Disk Drive	0xA1	BUSY
003	Network	0xC4	ERROR

The Dynamic Loop



- ✓ Initialized via Bootstrap.
- ✓ Governed via Resource Allocation.
- ✓ Reacts via Interrupts.

NEXT MODULE:
Process Management
& Scheduling.